

파이썬 리스트(List)

[1차시] 리스트 생성과 데이터 묶기

📖 핵심 개념 설명

학생별 점수나 지역별 인구처럼 같은 종류의 데이터를 여러 개 다룰 때 변수를 하나씩 만들면 관리하기 어렵습니다. 리스트는 대괄호 안에 여러 값을 순서대로 담는 자료형입니다. 데이터 분석 동아리의 설문 점수를 리스트로 묶고 자료의 개수를 확인합니다.

📄 전체 실습 코드

```
# 설문 점수를 순서대로 저장
scores = [82, 75, 90, 88, 75]
brands = ["세종우유", "한솔제과", "세종푸드"]
empty_data = []

print("설문 점수:", scores)
print("응답자 수:", len(scores))
print("분석 대상:", brands)
print("수집 전 데이터:", empty_data)
```

🔍 코드 훑아보기

- **대괄호 []** : 쉼표로 구분한 값들을 하나의 리스트로 만듭니다. 같은 값 75가 두 번 들어가도 각각의 원소로 저장됩니다.
- **len(scores)** : 리스트에 담긴 원소의 개수를 반환하므로 응답자 수는 5입니다.
- **빈 리스트 : []** : 원소가 없는 리스트입니다. 이후 데이터를 추가하는 출발점으로 사용할 수 있습니다.

[2차시] 인덱싱으로 필요한 값 꺼내기

📖 핵심 개념 설명

리스트의 각 원소에는 위치를 나타내는 인덱스가 있습니다. 첫 번째 위치는 1이 아니라 0이며 음수 인덱스는 뒤에서부터 셉니다. 월별 방문자 수에서 첫 달과 마지막 달의 값을 꺼내어 변화량을 계산합니다.

📄 전체 실습 코드

```
visitors = [120, 135, 128, 160]

first_month = visitors[0]
last_month = visitors[-1]
change = last_month - first_month

print("첫 달 방문자:", first_month)
print("두 번째 달 방문자:", visitors[1])
print("마지막 달 방문자:", last_month)
print("첫 달 대비 증가:", change)
# visitors[4]는 범위를 벗어나 IndexError가 발생합니다.
```

🔍 코드 훑아보기

- **visitors[0]** : 첫 번째 값 120을 가져옵니다. 길이가 4인 리스트의 양수 인덱스는 0부터 3까지입니다.
- **visitors[-1]** : 마지막 값 160을 가져옵니다. 자료가 추가되어도 마지막 원소를 가리킵니다.
- **change** : 꺼낸 값은 일반 숫자처럼 계산할 수 있으며 증가량은 40입니다.

[3차시] 슬라이싱으로 구간 추출하기

📖 핵심 개념 설명

인덱싱이 값 하나를 꺼내는 방법이라면 슬라이싱은 원하는 구간을 새 리스트로 만드는 방법입니다. [시작:끝]에서 끝 위치는 포함되지 않습니다. 일주일의 방문자 자료를 앞부분과 뒷부분으로 나누어 분석합니다.

📄 전체 실습 코드

```
visitors = [120, 135, 128, 160, 175, 210, 190]

first_three = visitors[:3]
weekend = visitors[5:7]
alternate = visitors[::2]
reversed_data = visitors[::-1]

print("첫 3일:", first_three)
print("주말:", weekend)
print("하루씩 건너뛴 자료:", alternate)
print("역순 자료:", reversed_data)
print("원본:", visitors)
```

🔍 코드 뜯아보기

- **visitors[:3]** : 시작을 생략하면 처음부터 가져옵니다. 인덱스 3은 제외되어 120, 135, 128이 선택됩니다.
- **visitors[5:7]** : 인덱스 5와 6의 값을 새 리스트에 담습니다. 주말 자료는 [210, 190]입니다.
- **간격과 역순** : 세 번째 값은 이동 간격입니다. ::2는 두 칸씩, ::-1은 뒤에서 앞으로 이동하며 원본은 바뀌지 않습니다.

[4차시] 원소 수정과 데이터 추가

📖 핵심 개념 설명

리스트는 만든 뒤에도 내용을 바꿀 수 있습니다. 인덱스로 기존 값을 수정하고 append로 값 하나를 마지막에 추가합니다. extend는 여러 원소를 풀어서 추가하고 insert는 지정한 위치에 값을 끼워 넣습니다. 설문 자료를 보완하며 세 방법을 비교합니다.

📄 전체 실습 코드

```
scores = [70, 80, 90]

scores[0] = 75
scores.append(85)
print("한 명 추가:", scores)

scores.extend([88, 92])
print("두 명 추가:", scores)

scores.insert(1, 78)
print("두 번째 위치에 삽입:", scores)
print("전체 응답자:", len(scores))
```

🔍 코드 뜯아보기

- **scores[0] = 75** : 잘못 입력한 첫 점수 70을 75로 교체합니다. 원소의 개수는 그대로입니다.
- **append와 extend** : append(85)는 숫자 하나를 추가합니다. extend([88, 92])는 두 숫자를 각각 추가합니다. append([88, 92])와 다릅니다.
- **insert(1, 78)** : 인덱스 1 앞에 78을 넣습니다. 뒤의 원소들은 한 칸씩 밀리며 최종 길이는 7입니다.

[5차시] 원소 삭제와 안전한 제거

📖 핵심 개념 설명

자료를 정리할 때는 값으로 지우는 방법과 위치로 지우는 방법을 구분해야 합니다. `remove`는 처음 일치하는 값 하나를 삭제하고 `pop`은 지정한 위치의 값을 꺼내면서 삭제합니다. 삭제할 값이 있는지 먼저 확인하면 불필요한 오류를 줄일 수 있습니다.

📄 전체 실습 코드

```
scores = [80, 0, 75, 0, 90]

if 0 in scores:
    scores.remove(0)
print("첫 번째 0 제거:", scores)

last_score = scores.pop()
print("꺼낸 마지막 점수:", last_score)
print("남은 자료:", scores)

del scores[0]
print("첫 원소 삭제:", scores)
```

🔍 코드 훑아보기

- **0 in scores** : 리스트에 0이 있는지 확인합니다. 없는 값을 `remove`로 지우면 `ValueError`가 발생합니다.
- **remove(0)** : 처음 발견한 0만 제거합니다. 모든 0을 제거하는 명령이 아니므로 다른 0은 남습니다.
- **pop과 del** : `pop()`은 마지막 값 90을 반환하며 삭제합니다. `del scores[0]`은 해당 위치를 삭제하고 최종 자료는 [75, 0]입니다.

[6차시] 검색과 정렬로 자료 살펴보기

📖 핵심 개념 설명

특정 값이 몇 번 나오는지 확인하거나 크기순으로 정리하면 자료의 특징을 쉽게 파악할 수 있습니다. `count`는 등장 횟수를 세고 `index`는 처음 나타난 위치를 찾습니다. `sorted`는 정렬한 새 리스트를 만들며 `sort`는 원본 자체를 정렬합니다.

📄 전체 실습 코드

```
scores = [82, 75, 90, 88, 75]

print("75점 인원:", scores.count(75))
if 90 in scores:
    print("90점의 첫 위치:", scores.index(90))

ranked = sorted(scores, reverse=True)
print("내림차순 새 리스트:", ranked)
print("정렬 전 원본:", scores)

scores.sort()
print("오름차순으로 바뀐 원본:", scores)
```

🔍 코드 훑아보기

- **count와 index** : 75는 두 번 나오고 90의 인덱스는 2입니다. 없는 값을 `index`로 찾으면 `ValueError`가 발생합니다.
- **sorted(..., reverse=True)** : 큰 값부터 정렬한 새 리스트 `[90, 88, 82, 75, 75]`를 만듭니다.
- **scores.sort()** : 원본을 오름차순으로 바꾸고 `None`을 반환합니다. `scores = scores.sort()`처럼 대입하지 않습니다.

[7차시] 반복문으로 합계와 평균 구하기

📌 핵심 개념 설명

리스트의 원소를 for 반복문으로 하나씩 꺼내면 같은 처리를 모든 자료에 적용할 수 있습니다. 합계를 저장할 변수를 0으로 준비한 뒤 각 값을 누적합니다. 이번에는 내장 함수의 결과와 직접 계산한 결과를 비교합니다.

📄 전체 실습 코드

```
populations = [18000, 32000, 25000, 15000]
total = 0

for population in populations:
    total = total + population

print("직접 계산한 합계:", total)
print("sum으로 구한 합계:", sum(populations))
if len(populations) > 0:
    average = total / len(populations)
    print("평균 인구:", average)
    print("최대 인구:", max(populations))
```

🔍 코드 훑아보기

- **total = 0** : 반복문 밖에서 누적 변수를 초기화합니다. 반복문 안에서 0으로 만들면 이전 합계가 지워집니다.
- **for population in populations** : 원소 네 개를 순서대로 꺼내어 총 90,000을 계산합니다.
- **빈 리스트 확인** : 자료가 없을 때 길이로 나누면 오류가 납니다. 이 예제의 평균은 22,500이며 max도 빈 자료에는 바로 사용할 수 없습니다.

[8차시] 조건에 맞는 자료와 리스트 컴프리헨션

🔍 핵심 개념 설명

분석에서는 전체 자료 중 조건에 맞는 값만 골라야 할 때가 많습니다. 반복문과 조건문으로 새 리스트를 만들 수 있으며 짧은 처리라면 리스트 컴프리헨션으로 표현할 수 있습니다. 80점 이상인 응답만 추출하고 점수를 보정합니다.

📄 전체 실습 코드

```
scores = [65, 82, 90, 73, 88]
high_scores = []

for score in scores:
    if score >= 80:
        high_scores.append(score)

compact_scores = [s for s in scores if s >= 80]
adjusted_scores = [min(s + 5, 100) for s in scores]

print("반복문으로 추출:", high_scores)
print("컴프리헨션으로 추출:", compact_scores)
print("5점 보정 결과:", adjusted_scores)
```

🔍 코드 톡아보기

- **조건과 append** : 80 이상인 값만 새 리스트에 추가하여 [82, 90, 88]을 얻습니다. 원본을 지우면서 반복하지 않습니다.
- **[s for s in scores if s >= 80]** : 원소를 꺼내고 조건을 확인한 뒤 결과에 넣는 작업을 한 줄로 표현합니다.
- **min(s + 5, 100)** : 5점을 더하되 100을 넘지 않게 합니다. 조건으로 거르는 일과 값을 바꾸는 일을 구분합니다.

[9차시] 중첩 리스트와 복사 이해하기

핵심 개념 설명

리스트 안에 리스트를 넣으면 행과 열이 있는 표처럼 자료를 다룰 수 있습니다. 중첩 리스트의 값은 인덱스를 두 번 사용하여 꺼냅니다.

전체 실습 코드

```
survey = [[80, 70], [90, 85], [75, 80]]
print("두 번째 응답자의 첫 점수:", survey[1][0])

for row in survey:
    print("응답자 평균:", sum(row) / len(row))

print("원본 첫 행:", survey[0])
```

코드 톺아보기

- `survey[1][0]` : 두 번째 행의 첫 번째 값 90입니다. 행과 열의 인덱스가 모두 0부터 시작합니다.

[10차시] 종합 미니 프로젝트 - 설문 점수 리포트

📌 핵심 개념 설명

지금까지 배운 추가, 조건 추출, 정렬, 집계를 연결하여 설문 점수 분석 프로그램을 완성합니다. 0부터 100까지를 유효한 점수로 정하고 범위 밖의 값은 제외합니다. 정리한 자료의 평균과 상위 점수, 기준 이상인 응답 수를 한 번에 출력합니다.

📄 전체 실습 코드

```
raw_scores = [82, 95, -1, 73, 88, 105, 90]
raw_scores.append(80)
valid_scores = [s for s in raw_scores if 0 <= s <= 100]

print("=== 설문 점수 분석 리포트 ===")
print("전체 수집:", len(raw_scores))
print("제외한 자료:", len(raw_scores) - len(valid_scores))

if valid_scores:
    average = sum(valid_scores) / len(valid_scores)
    top_three = sorted(valid_scores, reverse=True)[:3]
    high_scores = [s for s in valid_scores if s >= 80]
    print("평균:", round(average, 1))
    print("상위 3개 점수:", top_three)
    print("80점 이상 응답 수:", len(high_scores))
else:
    print("분석할 유효한 자료가 없습니다.")
```

🔍 코드 뜯아보기

- **유효 범위 설정** : -1과 105를 제외합니다. 실제 조사에서는 제외 기준을 먼저 정하고 제외한 개수도 함께 기록해야 합니다.
- **집계와 정렬 연결** : 유효 자료는 6개, 평균은 84.7점, 상위 점수는 [95, 90, 88], 80점 이상은 5개입니다.
- **직접 바꿔보기** : 기준을 85점으로 바꾸어 다시 실행하고 결과를 예측합니다. raw_scores를 모두 범위 밖의 값으로 바꾸어 빈 자료 처리도 확인합니다.